Introduction to Digital Communications

Sorbonne Université – Master SESI – UM4IN111 – PPSE

Adrien Cassagne

March 17, 2025

Table of Contents

- Introduction
- Simulation
- Monte Carlo Method
- Simulation Results
- Repeat-accumulate Codes

Context

1 Introduction

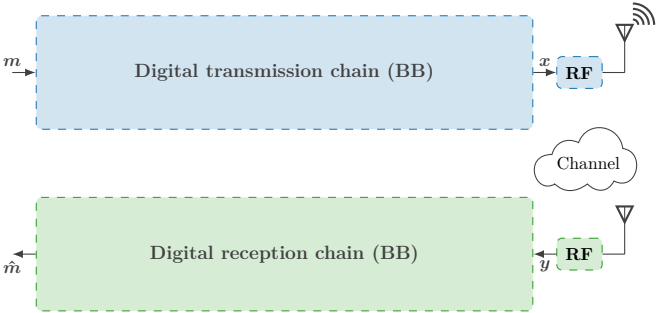
Ubiquitous digital communications, diverse usages & requirements

Radio Communication Systems

1 Introduction

- Transmitter (Tx) part & Receiver (Rx) part
- Physical separation between Tx and Rx: The channel
- Tx and Rx are separated in two main parts
  - Baseband (BB) or **digital signal processing**
  - Radio frequency (RF) or analog signal processing

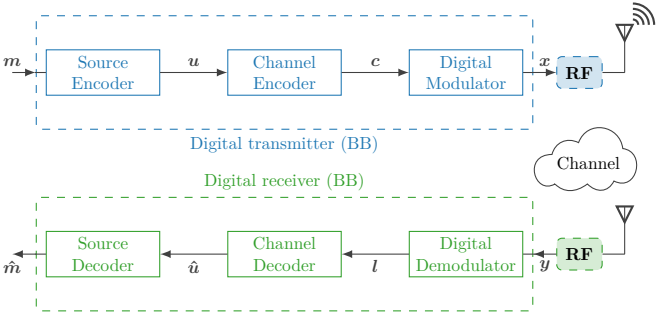
Digital Communication Systems



- Channel: Disturbances (noise, interferences)

Digital Communication Systems

1 Introduction



- Channel: Disturbances (noise, interferences)
- Source coding: Seek maximum conciseness in the expression of a message
- Channel coding: Make a message robust to channel disturbances

Open Systems Interconnection Model

OSI model:

Application (layer 7)
Presentation (layer 6)
Session (layer 5)
Transport (layer 4)
Network (layer 3)
Data Link (layer 2)
Physical (layer 1)

- We are considering digital communications at the **layer 1 of the OSI model**
- Traditionally the **physical layer is processed in hardware** by dedicated chips
- Generally **NOT a software area BUT**...

Open Systems Interconnection Model

OSI model:

Application (layer 7)
Presentation (layer 6)
Session (layer 5)
Transport (layer 4)
Network (layer 3)
Data Link (layer 2)
Physical (layer 1)

- We are considering digital communications at the **layer 1 of the OSI model**
- Traditionally the **physical layer is processed in hardware** by dedicated chips
- Generally **NOT a software area BUT**...
  - Software helps a lot when **designing dedicated chips** (ASIC/ASIP)
  - **Physical layer can be abstracted** and processed in software

Open Systems Interconnection Model

OSI model:

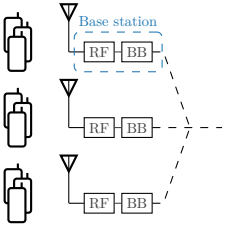
Application (layer 7)
Presentation (layer 6)
Session (layer 5)
Transport (layer 4)
Network (layer 3)
Data Link (layer 2)
Physical (layer 1)

- We are considering digital communications at the **layer 1 of the OSI model**
- Traditionally the **physical layer is processed in hardware** by dedicated chips
- Generally **NOT a software area BUT**...
  - Software helps a lot when **designing dedicated chips** (ASIC/ASIP)
  - **Physical layer can be abstracted** and processed in software
    - ➔ Less power efficient but **more flexible**
    - ➔ **New domain** and **opportunities**

Software-Defined Radio

Software-Defined Radio (SDR) = radio communication system all in software

Evolution of the mobile phone networks:

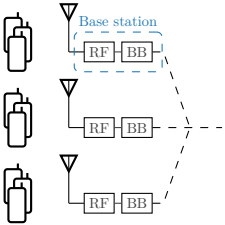


1G/2G

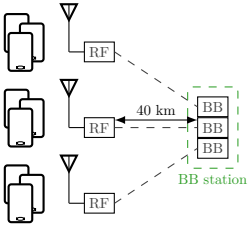
Software-Defined Radio

Software-Defined Radio (SDR) = radio communication system all in software

Evolution of the mobile phone networks:



1G/2G

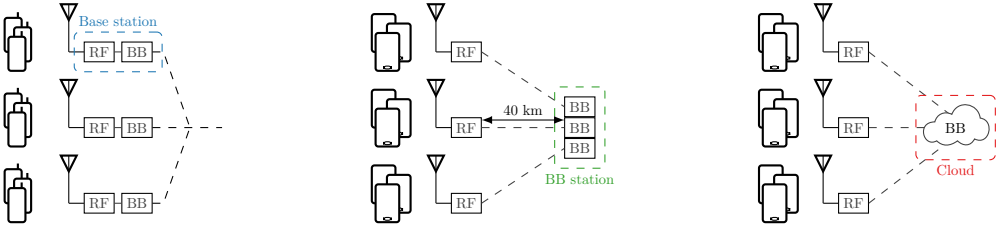


3G/4G

Software-Defined Radio

Software-Defined Radio (SDR) = radio communication system all in software

Evolution of the mobile phone networks:



1G/2G 3G/4G 5G and future

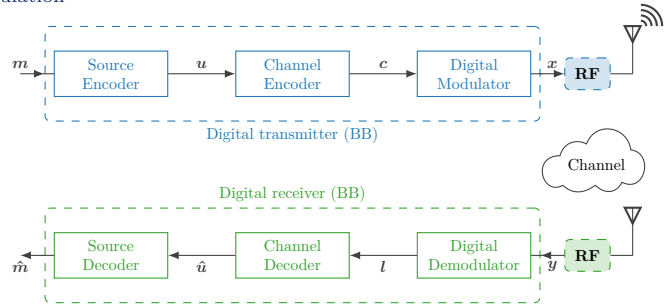
- BB virtualization = Cloud Radio Access Network (C-RAN)
- SDR considered for 5G

Table of Contents

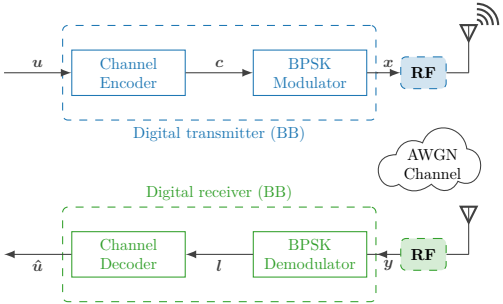
2 Simulation

- Introduction
- Simulation
- Monte Carlo Method
- Simulation Results
- Repeat-accumulate Codes

Functional Simulation (1)



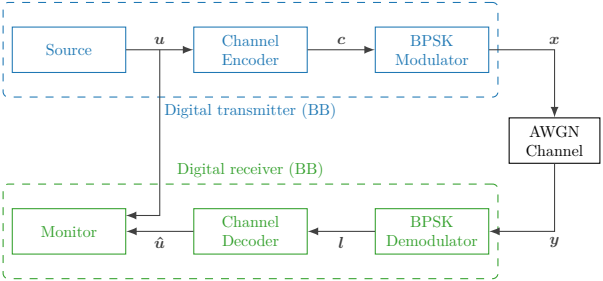
Functional Simulation (1)



- BPSK: Binary phase-shift keying modulation
- AWGN: Additive white Gaussian noise (normal distribution)

  ➔ Commonly used to evaluate error correcting codes

Functional Simulation (1)

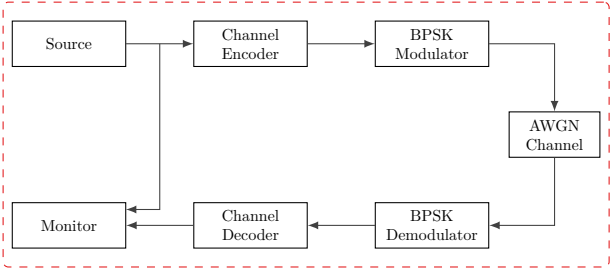


- BPSK: Binary phase-shift keying modulation
- AWGN: Additive white Gaussian noise (normal distribution)

  ➔ Commonly used to evaluate error correcting codes

- Source: Generates random frames
- Monitor: Compares initial message $\boldsymbol{u}$ with decoded message $\hat{\boldsymbol{u}}$

Functional Simulation (1)



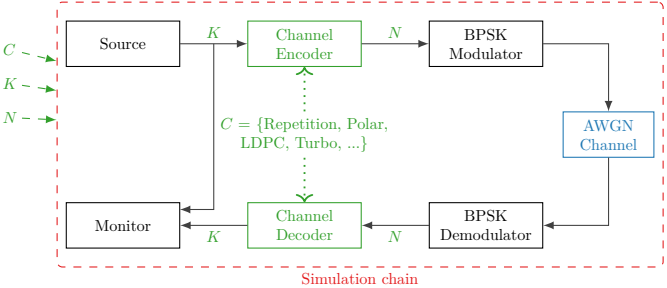
Simulation chain

- BPSK: Binary phase-shift keying modulation
- AWGN: Additive white Gaussian noise (normal distribution)

  ➔ Commonly used to evaluate error correcting codes

- Source: Generates random frames
- Monitor: Compares initial message $\boldsymbol{u}$ with decoded message $\hat{\boldsymbol{u}}$
- All the processing represents a **simulation chain**

Functional Simulation (2)

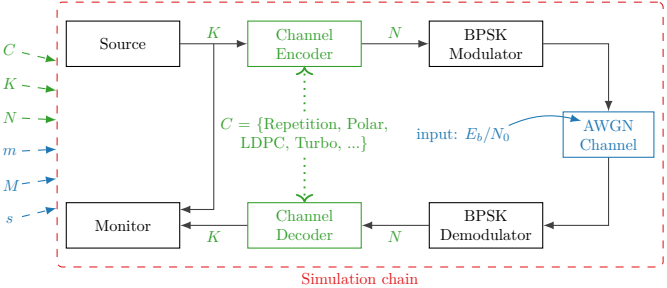


- BPSK: Binary phase-shift keying modulation
- AWGN: Additive white Gaussian noise (normal distribution)

  ➔ Commonly used to evaluate error correcting codes

- C, K, N: Characteristics of channel code

Functional Simulation (2)

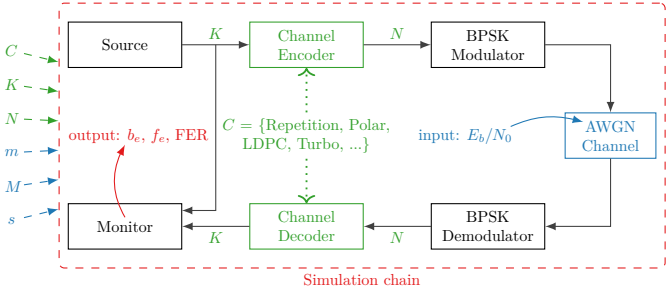


- BPSK: Binary phase-shift keying modulation
- AWGN: Additive white Gaussian noise (normal distribution)

  ➔ Commonly used to evaluate error correcting codes

- C, K, N: Characteristics of channel code
- m, M, s: Configurations of channel model

Functional Simulation (2)



- BPSK: Binary phase-shift keying modulation
- AWGN: Additive white Gaussian noise (normal distribution)

  ➔ Commonly used to evaluate error correcting codes

- C, K, N: Characteristics of channel code
- m, M, s: Configurations of channel model
- Frame Error Rate (FER) performance: estimated by the monitor

A Tour of the Chain – Random Source

- Generate K bits randomly to create a message $\boldsymbol{u}_K$
- Use of **pseudorandom number generator** (PRNG) with a **uniform distribution** between 0 and 1
- **A frame** is a sequence of K **information bits**
- For a frame K, there are 2^K possible sequences of bits
  - Example of a frame with $K = 4$: $\boldsymbol{u}_{K=4} = \{0, 1, 1, 0\}$

A Tour of the Chain – Repetition Encoding

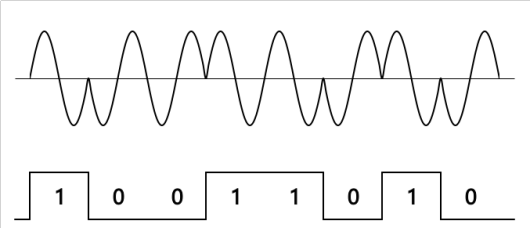
- **Intuitive** coding scheme: The frame $\boldsymbol{u}_K$ is **repeated** n times
- The generated bits are called a **codeword** $\boldsymbol{x}_N$
- A codeword contains N bits with $N \geq K$
- The **coderate** $R = K/N$ gives the ratio of the info. bits over the coded bits
  - $0 < R \leq 1$, the larger R, the less **redundancy**

A Tour of the Chain – Repetition Encoding

- **Intuitive** coding scheme: The frame $\boldsymbol{u}_K$ is **repeated** n times
- The generated bits are called a **codeword** $\boldsymbol{x}_N$
- A codeword contains N bits with $N \geq K$
- The **coderate** $R = K/N$ gives the ratio of the info. bits over the coded bits
  - $0 < R \leq 1$, the larger R, the less **redundancy**
- Let's encode $\boldsymbol{u}_{K=4} = \{0, 1, 1, 0\}$,
  $\text{enc}_{\text{rep}}(\boldsymbol{u}_{K=4}, 3) = \{\underbrace{0, 1, 1, 0}_{\text{rep1}}, \underbrace{0, 1, 1, 0}_{\text{rep2}}, \underbrace{0, 1, 1, 0}_{\text{rep3}}\} = \boldsymbol{c}_{N=12}$
  - Be careful, all bit sequences of size 12 are not codewords
  - Example: $\{0, 1, 0, 0, \quad 0, 1, 1, 0, \quad 0, 1, 1, 0\}$ is not a repetition codeword

A Tour of the Chain – BPSK Modulation (1)

- **Binary phase-shift keying** (BPSK) or 2-PSK
- The simplest digital modulation where **two possible symbols**: 0 and 1



In the digital part of communication systems (= BB), we don't generate the sine waves, this is the responsability of the radio frenquency (RF = analog processing).

A Tour of the Chain – BPSK Modulation (2)

- **Binary phase-shift keying** (BPSK) or 2-PSK
- The simplest digital modulation where **two possible symbols**: 0 and 1
- In this presentation we will consider that the modulation is combined with the **mapping**
  - 0 is mapped onto the $+1$ value
  - 1 is mapped onto the -1 value

A Tour of the Chain – BPSK Modulation (2)

- **Binary phase-shift keying** (BPSK) or 2-PSK
- The simplest digital modulation where **two possible symbols**: 0 and 1
- In this presentation we will consider that the modulation is combined with the **mapping**
  - 0 is mapped onto the $+1$ value
  - 1 is mapped onto the -1 value
- Let's modulate $\boldsymbol{c}_{N=12} = \{0, 1, 1, 0, \quad 0, 1, 1, 0, \quad 0, 1, 1, 0\}$:

$$\text{mod}_{\text{BPSK}}(\boldsymbol{c}_{N=12}) = \{\underbrace{+1, -1, -1, +1}_{\text{rep1}}, \underbrace{+1, -1, -1, +1}_{\text{rep2}}, \underbrace{+1, -1, -1, +1}_{\text{rep3}}\} = \boldsymbol{x}_{N=12}$$

- This is the **last digital step** of the Tx

A Tour of the Chain – AWGN Channel (1)

- The channel **simulates the noise/perturbations** that occurs on the produced symbols when they are transmitted
- The noise can have various origins in radio communications
- Additive White Gaussian Noise (AGWN)
  - Models the **thermal noise of the Rx circuit**
    "Thermal noise is the noise produced by the random motion of charged particles (usually electrons) in conducting media"
  - $\boldsymbol{y} = \boldsymbol{x} + \boldsymbol{n}$ (where $\boldsymbol{n}$ is a normally distributed noise)
  - $\boldsymbol{y}$ are noisy symbols
  - The **standard deviation** σ determines the amount of noise

A Tour of the Chain – AWGN Channel (2)

2 Simulation

- y_i output is bound to its x_i input in the form $y_i = x_i + n_i$ with N_{chn} an **independent and identically distributed variance** according to a **normal law** centered in zero
- The variance is $\sigma^2 = N_0/2$ and $N_{\text{chn}} \simeq \mathcal{N}(0, \sigma^2)$, and we have:

$$P(y_i|x_i) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(y_i - x_i)^2}{2\sigma^2}\right). \tag{1}$$

- It is very common to vary **Signal-to-Noise Ratio** (SNR), given by
  - E_b/N_0 (in dB) where E_b is the average energy per information bit or
  - E_s/N_0 (in dB) where E_s is the average energy per transmitted symbol

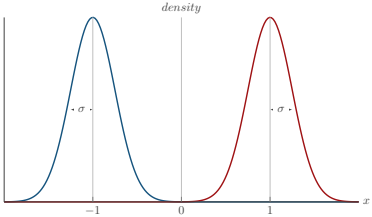
$$\frac{E_s}{N_0} = \frac{E_b}{N_0} + 10.\log_{10}(R.b_S) \text{ with } b_s \text{ the number of bits of a symbol.} \tag{2}$$

A Tour of the Chain – AWGN Channel (3)

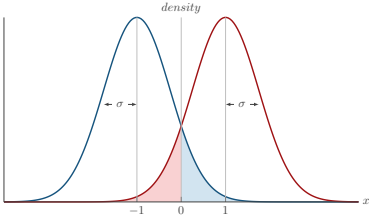
2 Simulation

- Considering E_s for the SNR, the channel variance is: $\sigma = \sqrt{\frac{1}{2\times 10^{\frac{E_s}{N_0}/10}}}$.

$\sigma = 0.25$, **NO** possible error



$\sigma = 0.75$, **possible errors** in blue and red areas



A Tour of the Chain – BPSK Demodulation

- Convert from noisy symbols to **Log Likelihood Ratio** (LLR) as follow:

$$l_i = \log\left(\frac{P(y_i|c_i = 0)}{P(y_i|c_i = 1)}\right).$$

  - A LLR is a real number that represents the probability of a bit to be 0 or 1
  - A negative LLR represents a number that most likely comes from a binary 1.
  - A positive LLR represents a number that most likely comes from a binary 0.

- Re-scaling $\boldsymbol{y}_N$ is required to obtain LLRs (σ-independent, specific to AWGN channel):

$$l_i = \frac{2 \times y_i}{\sigma^2}.$$

A Tour of the Chain – Rep. Hard Decoding

- Take an **hard decision** (= convert l_i into bits $l_i^h = h(l_i)$): $l_i^h = \begin{cases} 0, & \text{if } l_i \geq 0, \\ 1, & \text{otherwise.} \end{cases}$
- We received $\boldsymbol{l}_{N=12} = \{\underbrace{0.2, -1.3, 0.3, 1.2}_{\text{rep1}}, \underbrace{0.8, -0.5, -1.5, 0.4}_{\text{rep2}}, \underbrace{-0.1, 0.2, -1.8, 2}_{\text{rep3}}\}$
- Hard decision gives: $\boldsymbol{l}_{N=12}^h = \{0, 1, 0, 0, \quad 0, 1, 1, 0, \quad 1, 0, 1, 0\}$

A Tour of the Chain – Rep. Hard Decoding

- Take an **hard decision** (= convert l_i into bits $l_i^h = h(l_i)$): $l_i^h = \begin{cases} 0, & \text{if } l_i \geq 0, \\ 1, & \text{otherwise.} \end{cases}$
- We received $\boldsymbol{l}_{N=12} = \{\underbrace{0.2, -1.3, 0.3, 1.2}_{\text{rep1}}, \underbrace{0.8, -0.5, -1.5, 0.4}_{\text{rep2}}, \underbrace{-0.1, 0.2, -1.8, 2}_{\text{rep3}}\}$
- Hard decision gives: $\boldsymbol{l}_{N=12}^h = \{0, 1, 0, 0, \quad 0, 1, 1, 0, \quad 1, 0, 1, 0\}$
- Then we can make a **majority vote**: let's count 1 as `-1` et 0 as `+1`

$$\boldsymbol{l}^{\text{vote}} = \{1, -1, -1, 3\}$$

- A negative number means the bit is 1, while a positive number means the bit is 0

$$\hat{\boldsymbol{u}}_{K=4} = h(\boldsymbol{l}^{\text{vote}}) = \{0, 1, 1, 0\}$$

A Tour of the Chain – Rep. Soft Decoding

- We received $\boldsymbol{l}_{12} = \{\underbrace{0.2, -1.3, 0.3, 1.2}_{\text{rep1}}, \underbrace{0.8, -0.5, -1.5, 0.4}_{\text{rep2}}, \underbrace{-0.1, 0.2, -1.8, 2}_{\text{rep3}}\}$
- We can compute average for each bit

$$\boldsymbol{l}^{\text{avg}} = \{0.2+0.8-0.1, -1.3-0.5+0.2, 0.3-1.5-1.8, 1.2+0.4+2\} = \{0.9, -1.6, -3, 3.6\}$$

- And finally we can make an hard decision on $\boldsymbol{l}^{\text{avg}}$

$$\hat{\boldsymbol{u}}_{K=4} = h(\{0.8, -1.6, -3, 3.6\}) = \{0, 1, 1, 0\}$$

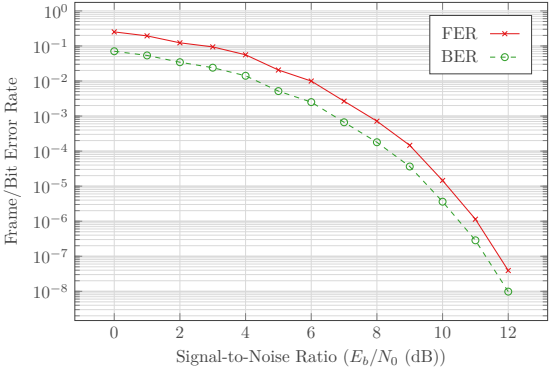
- To your opinion, which one is the best, hard or soft decoder?

A Tour of the Chain – Monitor (1)

- Compare $\boldsymbol{u}_K$ and $\hat{\boldsymbol{u}}_K$
  - If all the bits are the same, then there is no bit error and no frame error
- Let's suppose we have $\boldsymbol{u}_{K=4} = \{0, 1, 1, 0\}$ and $\hat{\boldsymbol{u}}_{K=4} = \{1, 0, 1, 0\}$
  - It means that the frame is wrong (= one frame error)
  - And we have two wrongs bits in the frame (= two bit errors)
- The monitor basically **counts the number of frame errors** f_e **and the number of bit errors** b_e when many frames are generated
- And from these two previous metrics we can estimate:
  - The **Frame Error Rate** (FER): $f_\mathrm{e}/f_\mathrm{n}$
  - The **Bit Error Rate** (BER): $b_\mathrm{e}/(f_\mathrm{n} \times K)$
    with f_n the numbers of simulated frames

A Tour of the Chain – Monitor (2)

2 Simulation



$K = 4$ and $N = 12$ repetition code (soft decoder) with BPSK modem and AWGN channel.

Table of Contents

3 Monte Carlo Method

- Introduction
- Simulation
- Monte Carlo Method
- Simulation Results
- Repeat-accumulate Codes

Monte Carlo Method

- Based on **repeated random sampling** to obtain numerical results
- Underlying concept is to use **randomness to solve problems**
- Main steps
  1. Define a domain of possible inputs
  2. Generate inputs randomly from a probability distribution over the domain
  3. Perform a deterministic computation of the outputs
  4. Aggregate the results

Applying it to Digital Communications

1. Domain of possible inputs is 2^K different frames of information bits
2. Inputs (symbols) are randomly noised following a normal law in the channel
3. Demodulation and decoding are deterministic computations and generates $\hat{\boldsymbol{u}}_K$ decoded bits
4. Monitor aggregates the results to compute frame error rate and bit error rate metrics

- For each noise level (= SNR point), f_n frames are simulated and the simulation stops when 100 frames with at least one error inside are detected ($f_{\max} = 100$)
  - f_n is unknown at the beginning of the simulation of the SNR point

Main Monte Carlo Algorithm

Algorithm 1: Functionnal simulation of a digital communication system.

Data: A list of SNRs, the number of information bits K, the codeword size N

$f_{\max} \leftarrow 100$;
foreach $\frac{E_b}{N_0}$ *in the SNRs* **do** ▷ loop over the SNRs
  $\{b_e, f_e, f_n\} \leftarrow 0$; ▷ b_e: bit errors, f_e: fra errors, f_n: sim frames
  $\frac{E_s}{N_0} \leftarrow \frac{E_b}{N_0} + 10.\log_{10}(R.b_s)$; ▷ here $b_s = 1$ (BPSK) and $R = K/N$
  $\sigma \leftarrow \sqrt{\frac{1}{2\times 10^{\frac{E_s}{N_0}/10}}}$;
  do ▷ loop over the frames, f_n depends on SNR, code, modem and channel
    $\boldsymbol{u}_K \leftarrow$ *Source* – generate();
    $\boldsymbol{c}_N \leftarrow$ *Repetition* – encode($\boldsymbol{u}_K$);
    $\boldsymbol{x}_N \leftarrow$ *BPSK* – modulate($\boldsymbol{c}_N$);
    $\boldsymbol{y}_N \leftarrow$ *AWGN* – add noise($\boldsymbol{x}_N, \sigma$) ; ▷ samples are noised here
    $\boldsymbol{l}_N \leftarrow$ *BPSK* – demodulate($\boldsymbol{y}_N, \sigma$);
    $\hat{\boldsymbol{u}}_K \leftarrow$ *Repetition* – decode($\boldsymbol{l}_N$);
    $\{b_e, f_e\} \leftarrow \{b_e, f_e\}$ + *Monitor* – check errors($\boldsymbol{u}_K, \hat{\boldsymbol{u}}_K$);
    $f_n \leftarrow f_n + 1$;
  while $f_e < f_{max}$;
  output $b_e/(f_n \times K)$; ▷ bit error rate for current $\frac{E_b}{N_0}$
  output f_e/f_n ; ▷ frame error rate for current $\frac{E_b}{N_0}$
end

Limitations and Possible Solutions

Problem:

- Simulating high SNRs can take a while (from hours to days...)
  - Because the number of frames we need to simulate (f_n) becomes very large before to reach 100 frame errors ($f_e < f_{\max}$)

Solutions:

- Find an other way to estimate the decoding performance of the system
  - Sometimes possible for simple channel, modulation and coding schemes
- Optimize the code to speedup the throughput of the simulation
  - Reduce the latency to generate and simulate one frame
  - Simulate many frames in parallel (= increase the throughput)

➔ **Clearly one of the main purpose of this class ;-)**

Table of Contents

4 Simulation Results

- Introduction
- Simulation
- Monte Carlo Method
- Simulation Results
- Repeat-accumulate Codes

List of Considered Simulations

- $\mathcal{S}im_1$:

```
./simulator -m 0 -M 15 -s 1 -e 100 -K 32 -N 128 -D "rep-hard" # R = 32/128 = 1/4
```

- $\mathcal{S}im_2$:

```
./simulator -m 0 -M 12 -s 1 -e 100 -K 32 -N 128 -D "rep-soft" # R = 32/128 = 1/4
```

- $\mathcal{S}im_3$:

```
./simulator -m 0 -M 12 -s 1 -e 100 -K 32 -N  96 -D "rep-soft" # R = 32/96 = 1/3
```

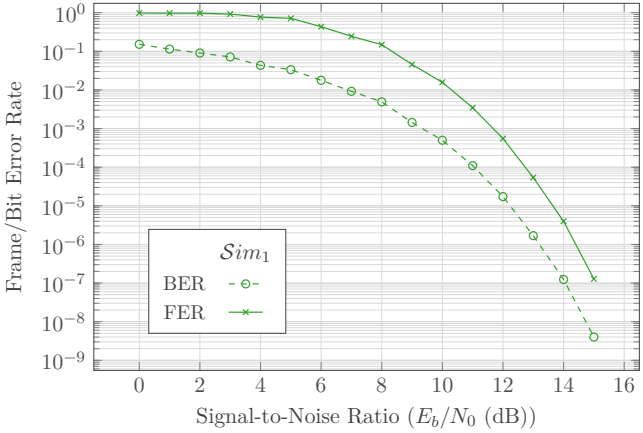
- $\mathcal{S}im_4$:

```
./simulator -m 0 -M 12 -s 1 -e 100 -K 32 -N  64 -D "rep-soft" # R = 32/64 = 1/2
```

- $\mathcal{S}im_5$:

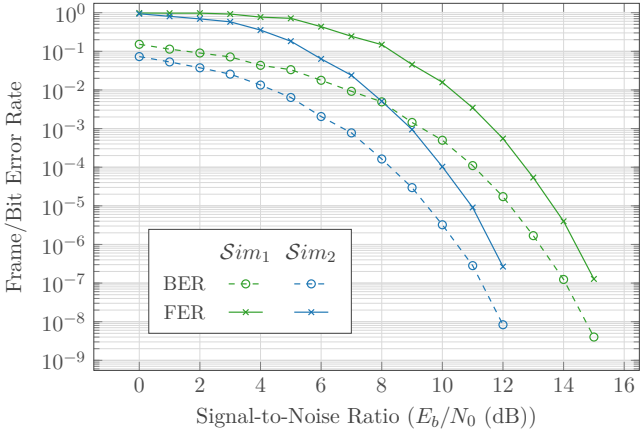
```
./simulator -m 0 -M 12 -s 1 -e 100 -K 32 -N  32 -D "rep-soft" # R = 32/32 = 1 => UNCODED!
```

Hard Input Decoder VS Soft Input Decoder



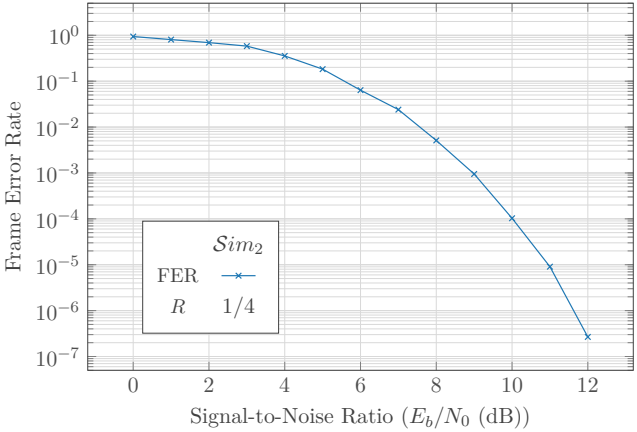
$K = 32$ and $N = 128$ repetition code (soft & hard decoders) with BPSK modem and AWGN channel.

Hard Input Decoder VS Soft Input Decoder



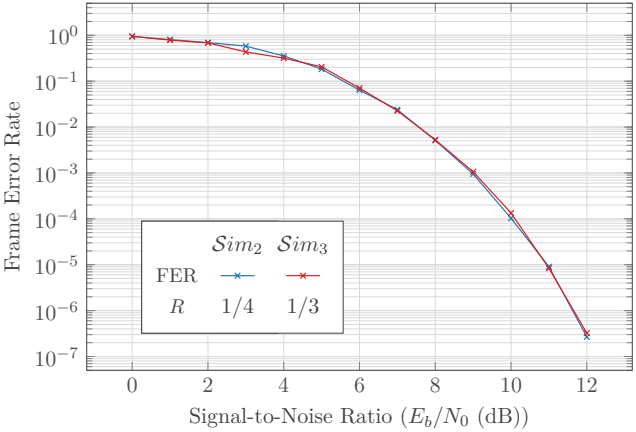
$K = 32$ and $N = 128$ repetition code (soft & hard decoders) with BPSK modem and AWGN channel.

Various Code Rates depending on E_b/N_0



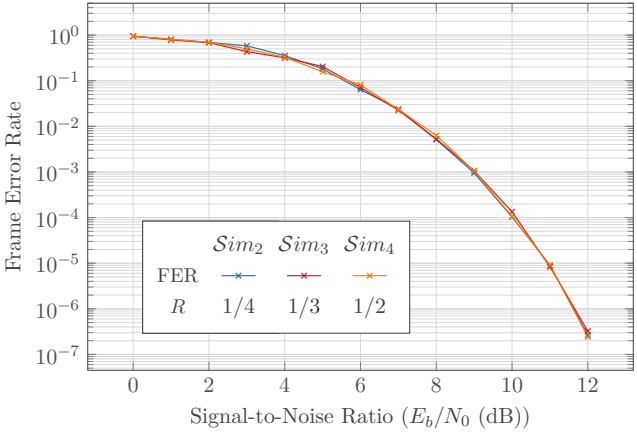
$K = 32$ repetition code (soft decoder) with BPSK modem and AWGN channel.

Various Code Rates depending on E_b/N_0



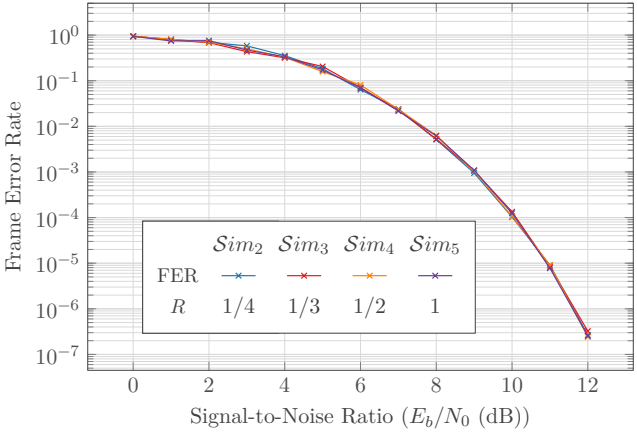
$K = 32$ repetition code (soft decoder) with BPSK modem and AWGN channel.

Various Code Rates depending on E_b/N_0



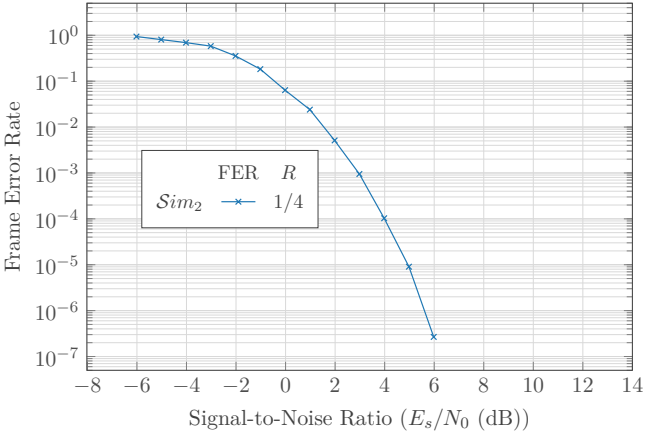
$K = 32$ repetition code (soft decoder) with BPSK modem and AWGN channel.

Various Code Rates depending on E_b/N_0



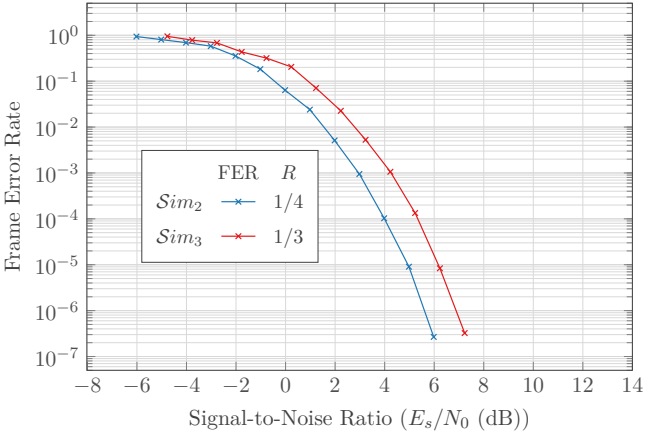
$K = 32$ repetition code (soft decoder) with BPSK modem and AWGN channel.

Various Code Rates depending on E_s/N_0



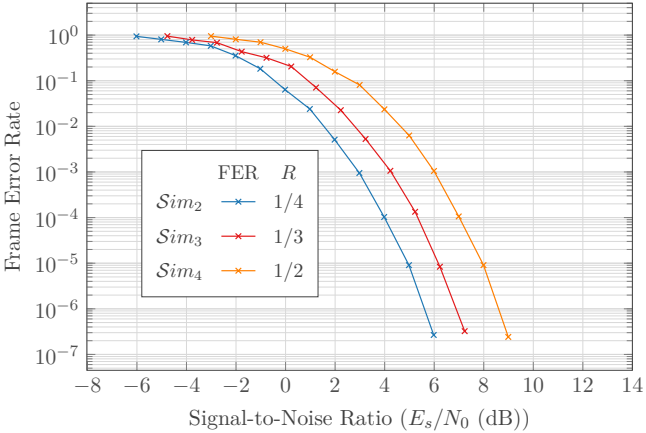
$K = 32$ repetition code (soft decoder) with BPSK modem and AWGN channel.

Various Code Rates depending on E_s/N_0



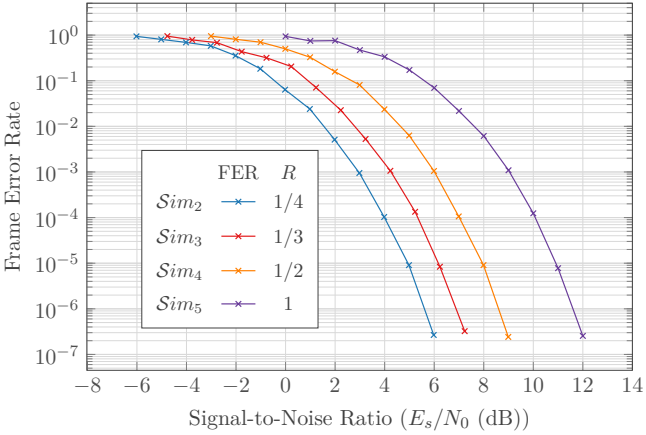
$K = 32$ repetition code (soft decoder) with BPSK modem and AWGN channel.

Various Code Rates depending on E_s/N_0



$K = 32$ repetition code (soft decoder) with BPSK modem and AWGN channel.

Various Code Rates depending on E_s/N_0



$K = 32$ repetition code (soft decoder) with BPSK modem and AWGN channel.

Table of Contents

5 Repeat-accumulate Codes

- Introduction
- Simulation
- Monte Carlo Method
- Simulation Results
- Repeat-accumulate Codes

Introduction

5 Repeat-accumulate Codes

Repeat-accumulate codes, also known as RA codes[1]

- Low complexity class of error-correcting codes
  - Like repetition codes
- Introduced to prove AWGN coding theorems targeting complex Turbo codes[2]
  - Thus, RA codes are a subclass of Turbo and LDPC codes[3]
- Native code rates are $R = 1/q$ with q the number of repetitions
  - Like repetition codes

[1] Dariush Divsalar, Hui Jin, and Robert J. McEliece. "Coding Theorems for "Turbo-Like" Codes". In: *Allerton Conference on Communications, Control, and Computing*. 1998. URL: https://www.researchgate.net/profile/Dariush-Divsalar/publication/243768674_Coding_Theorems_for_Turbo-Like_Codes/links/53e3cc5c0cf2fb74870dbbc1/Coding-Theorems-for-Turbo-Like-Codes.pdf.

[2] C. Berrou, A. Glavieux, and P. Thitimajshima. "Near Shannon Limit Error-Correcting Coding and Decoding: Turbo-Codes". In: *International Conference on Communications (ICC)*. vol. 2. IEEE, May 1993, pp. 1064–1070. DOI: 10.1109/ICC.1993.397441.

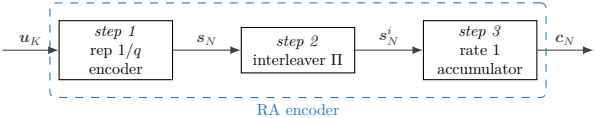
[3] R. Gallager. "Low-Density Parity-Check Codes". In: *IRE Transactions on Information Theory* 8.1 (Jan. 1962), pp. 21–28. DOI: 10.1109/TIT.1962.1057683.

Coding Scheme – Encoder (1)

1. K information bits are repeated q times, resulting in a bit sequence $\boldsymbol{s}_N$ of length $N = q \times K$
2. A random (but fixed) permutation is applied to the resulting bit sequence $\boldsymbol{s}_N$ producing $\boldsymbol{s}_N^i$ interleaved sequence of bits
3. The $\boldsymbol{s}_N^i$ interleaved sequence of bits is fed to a rate-1 accumulator – with $1/(1+D)$ transfer function – producing the $\boldsymbol{c}_N$ codeword

$$
\begin{aligned}
c_0 &= s_0^i \\
c_1 &= s_0^i \oplus s_1^i \\
c_2 &= s_0^i \oplus s_1^i \oplus s_2^i \\
&\dots \\
c_{N-1} &= s_0^i \oplus s_1^i \oplus s_2^i \oplus \dots \oplus s_{N-1}^i
\end{aligned}
$$

Coding Scheme – Encoder (2)



Example with $K = 4$ and $q = 3$

1. $\boldsymbol{u}_{K=4} = \{0, 1, 1, 0\} \rightarrow \boldsymbol{s}_{N=12} = \{0, 1, 1, 0, \quad 0, 1, 1, 0, \quad 0, 1, 1, 0\}$
2. Let's consider the following permutation indices
   $\boldsymbol{p}_{N=12} = \{0, 4, 8, \quad 1, 5, 9, \quad 2, 6, 10, \quad 3, 7, 11\}$, the interleaver Π will produce
   $\boldsymbol{s}^i_{N=12} = \{0, 0, 0, \quad 1, 1, 1, \quad 1, 1, 1, \quad 0, 0, 0\}$
   - In this class we will call this interleaver a **transpose** interleaver
   - The first dimension is K and the second one is q

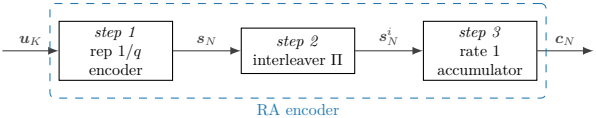
$$T(\boldsymbol{s}_{[K,q]}) = \boldsymbol{s}^i_{[q,K]} = \boldsymbol{s}^i_N. \tag{3}$$

Coding Scheme – Encoder (3)

3. From $\boldsymbol{s}^i_{N=12} = \{0,0,0,\ \ 1,1,1,\ \ 1,1,1,\ \ 0,0,0\}$ the rate-1 accumulator produces:

$$
\begin{aligned}
\boldsymbol{c}_{N=12} = \{ & 0 && = 0 \\
& 0 \oplus 0 && = 0 \\
& 0 \oplus 0 \oplus 0 && = 0 \\
& 0 \oplus 0 \oplus 0 \oplus 1 && = 1 \\
& 0 \oplus 0 \oplus 0 \oplus 1 \oplus 1 && = 0 \\
& 0 \oplus 0 \oplus 0 \oplus 1 \oplus 1 \oplus 1 && = 1 \\
& 0 \oplus 0 \oplus 0 \oplus 1 \oplus 1 \oplus 1 \oplus 1 && = 0 \\
& 0 \oplus 0 \oplus 0 \oplus 1 \oplus 1 \oplus 1 \oplus 1 \oplus 1 && = 1 \\
& 0 \oplus 0 \oplus 0 \oplus 1 \oplus 1 \oplus 1 \oplus 1 \oplus 1 \oplus 1 && = 0 \\
& 0 \oplus 0 \oplus 0 \oplus 1 \oplus 1 \oplus 1 \oplus 1 \oplus 1 \oplus 1 \oplus 0 && = 0 \\
& 0 \oplus 0 \oplus 0 \oplus 1 \oplus 1 \oplus 1 \oplus 1 \oplus 1 \oplus 1 \oplus 0 \oplus 0 && = 0 \\
& 0 \oplus 0 \oplus 0 \oplus 1 \oplus 1 \oplus 1 \oplus 1 \oplus 1 \oplus 1 \oplus 0 \oplus 0 \oplus 0 && = 0\}
\end{aligned}
$$

Coding Scheme – Encoder (4)



Example with $K = 4$ and $q = 3$

1. $\boldsymbol{u}_{K=4} = \{0, 1, 1, 0\} \rightarrow \boldsymbol{s}_{N=12} = \{0, 1, 1, 0, \quad 0, 1, 1, 0, \quad 0, 1, 1, 0\}$
2. Let's consider the following permutation indices
   $\boldsymbol{p}_{N=12} = \{0, 4, 8, \quad 1, 5, 9, \quad 2, 6, 10, \quad 3, 7, 11\}$, the interleaver Π will produce
   $\boldsymbol{s}^i_{N=12} = \{0, 0, 0, \quad 1, 1, 1, \quad 1, 1, 1, \quad 0, 0, 0\}$
3. From $\boldsymbol{s}^i_{N=12} = \{0, 0, 0, \quad 1, 1, 1, \quad 1, 1, 1, \quad 0, 0, 0\}$ the rate-1 accumulator produces
   $\boldsymbol{c}_{N=12} = \{0, 0, 0, 1, 0, 1, 0, 1, 0, 0, 0, 0\}$
   — This is a non-systematic code (the K information bits are not "visible" directly)

Getting Back Info. Bits from a Codeword

Let's suppose we know $\boldsymbol{c}_N$ and we want to find back $\boldsymbol{u}_K$.

We have:

$$
\begin{aligned}
c_0 &= s_0^i \\
c_1 &= s_0^i \oplus s_1^i \\
c_2 &= s_0^i \oplus s_1^i \oplus s_2^i \\
&\dots \\
c_{N-1} &= s_0^i \oplus s_1^i \oplus s_2^i \oplus \dots \oplus s_{N-1}^i
\end{aligned}
$$

And we can rewrite it as:

$$
\begin{aligned}
s_0^i &= c_0 \\
s_1^i &= c_0 \oplus c_1 \\
s_2^i &= c_1 \oplus c_2 \\
&\dots \\
s_{N-1}^i &= c_{N-2} \oplus c_{N-1}
\end{aligned}
$$

Then, we need to deinterleave the $\boldsymbol{s}_N^i$ bits with Π^{-1} and we will get $\boldsymbol{s}_N$.

Finally we just need to read the first K bits of $\boldsymbol{s}_N$ to retrieve $\boldsymbol{u}_K$.

Some Prerequisites

- In the class we are only considering binary error correcting codes
- Binary codes are defined in GALOIS field of two elements (GF_2)

a	b	$a \oplus b$	ab
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Elementary operations in GF_2 (logical *exclusive or* and logical *and*).

- When working on LLR (log domain), $a \oplus b$ operator transforms into:

$$l_a \boxplus l_b = \underbrace{2\tanh^{-1}\left(\tanh\left(\frac{l_a}{2}\right).\tanh\left(\frac{l_b}{2}\right)\right)}_{\text{Sum-Product Algorithm (SPA)}} \approx \underbrace{\text{sign}(l_a.l_b).\min(|l_a|,|l_b|)}_{\text{Min-Sum Approx. (MS)}}.$$

- The multiplication in GF_2 transforms into a simple addition in log domain

Single Iteration Soft Input Decoder

$\tau_0^u \leftarrow l_0$;
for $n \leftarrow 1$ *to* $N-1$ **do** ▷ `decode rate-1 accu.`
  $\tau_n^u \leftarrow l_{n-1} \boxplus l_n$;
end
$\boldsymbol{\omega}_N^u \leftarrow \Pi^{-1}(\boldsymbol{\tau}_N^u)$; ▷ `deinterleave`
for $k \leftarrow 0$ *to* $K-1$ **do** ▷ `decode repetition`
  $\hat{u}_k \leftarrow h(\sum_{r \leftarrow 0}^{r \leq q-1} \omega_{r \times K+k}^u)$;
end

- Input
  - $\boldsymbol{l}_N$ the N LLRs from the demodulator
- Internal buffers
  - $\boldsymbol{\tau}_N^u$ and $\boldsymbol{\omega}_N^u$ real buffers of size N
- Output
  - $\hat{\boldsymbol{u}}_K$ the decided bits of size K

Iterative Soft Input Decoder

5 Repeat-accumulate Codes

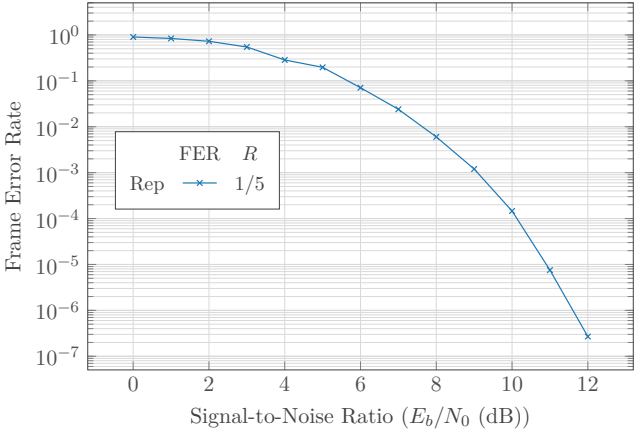
$\{\boldsymbol{\alpha}_N, \boldsymbol{\beta}_N, \boldsymbol{\tau}_N^d\} \leftarrow 0;$
for $i \leftarrow 0$ *to* $i_{max} - 1$ **do** ▷ loop over the iterations
  $\alpha_0 \leftarrow \tau_0^d;$
  for $n \leftarrow 1$ *to* $N-1$ **do** ▷ forward pass
    $\alpha_n \leftarrow (\alpha_{n-1} + l_{n-1}) \boxplus \tau_n^d;$
  end
  $\beta_{N-2} \leftarrow l_{N-1} \boxplus \tau_{N-1}^d;$
  for $n \leftarrow N-3$ *to* 0 **do** ▷ backward pass
    $\beta_n \leftarrow (\beta_{n+1} + l_{n+1}) \boxplus \tau_{n+1}^d;$
  end
  $\tau_0^u \leftarrow \beta_0 + l_0;\ \tau_{N-1}^u \leftarrow (\alpha_{N-2} + l_{N-2}) \boxplus l_{N-1};$
  for $n \leftarrow 1$ *to* $N-2$ **do** ▷ compute extrinsic
    $\tau_n^u \leftarrow (\alpha_{n-1} + l_{n-1}) \boxplus (\beta_n + l_n);$
  end
  $\boldsymbol{\omega}_N^u \leftarrow \Pi^{-1}(\boldsymbol{\tau}_N^u)\ ;$ ▷ deinterleave
  for $k \leftarrow 0$ *to* $K-1$ **do** ▷ repetition soft decode
    $l_k^{\text{out}} \leftarrow \sum_{r \leftarrow 0}^{r \leq q-1} \omega_{r \times K + k}^u;$
    for $r \leftarrow 0$ *to* $q-1$ **do** ▷ remove self contribution
      $\omega_{r \times K + k}^d \leftarrow l_k^{\text{out}} - \omega_{r \times K + k}^u;$
    end
  end
  $\boldsymbol{\tau}_N^d \leftarrow \Pi(\boldsymbol{\omega}_N^d)\ ;$ ▷ interleave for next iteration
end
$\hat{\boldsymbol{u}}_K \leftarrow h(\boldsymbol{l}_K^{\text{out}})\ ;$ ▷ take final hard decision

- Inputs
  - $\boldsymbol{l}_N$ the N LLRs from the demodulator
  - $i_{\max}$ the number of iterations to perform
- Internal buffers
  - $\boldsymbol{\alpha}_N$, $\boldsymbol{\beta}_N$, $\boldsymbol{\tau}_N^d$, $\boldsymbol{\tau}_N^u$, $\boldsymbol{\omega}_N^u$ and $\boldsymbol{\omega}_N^d$ real buffers of size N
  - $\boldsymbol{l}_K^{\text{out}}$ a real buffer of size K, represent the soft estimation of $\hat{\boldsymbol{u}}_K$
- Output
  - $\hat{\boldsymbol{u}}_K$ the decided bits of size K

Comments on Decoders

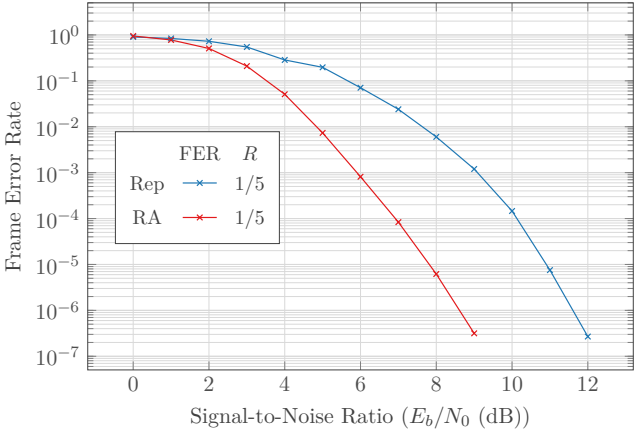
- Decoding performance on one iteration is expected to be worse than simple repetition codes
- Iterative algorithm (when $i_{max} \geq 2$) outperforms repetition codes

Various Code Rates depending on E_b/N_0



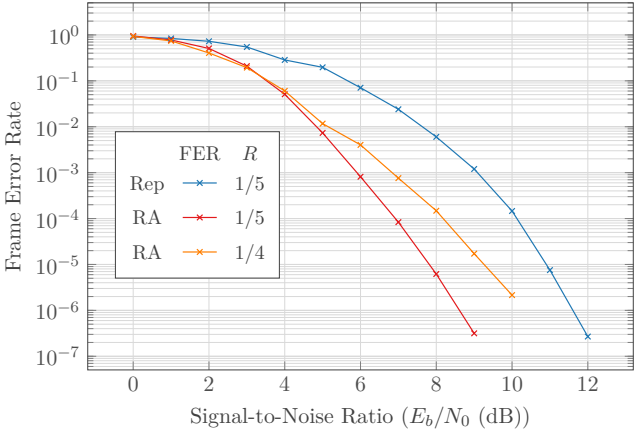
$K = 32$ RA code with random interleaver and 4 iterations.

Various Code Rates depending on E_b/N_0



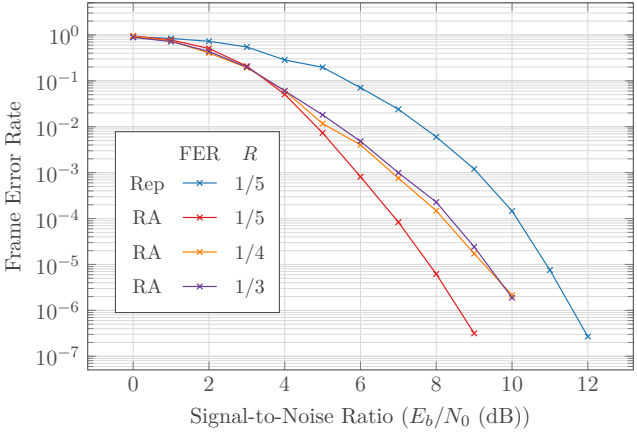
$K = 32$ RA code with random interleaver and 4 iterations.

Various Code Rates depending on E_b/N_0



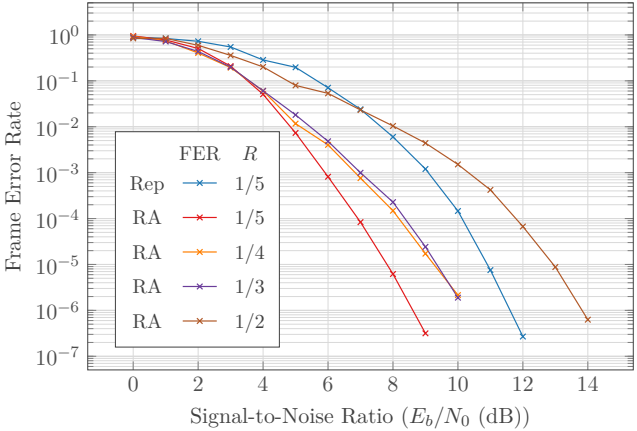
$K = 32$ RA code with random interleaver and 4 iterations.

Various Code Rates depending on E_b/N_0



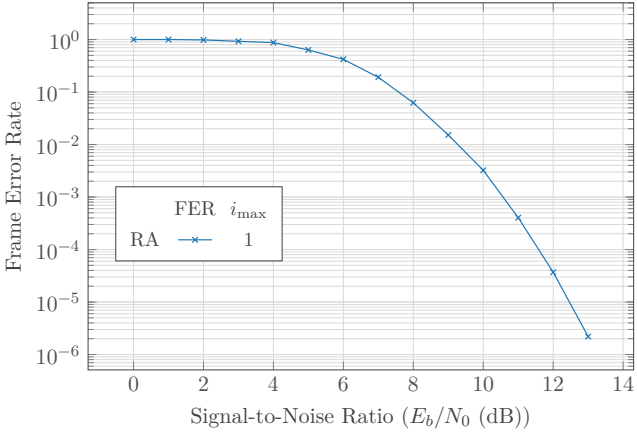
$K = 32$ RA code with random interleaver and 4 iterations.

Various Code Rates depending on E_b/N_0



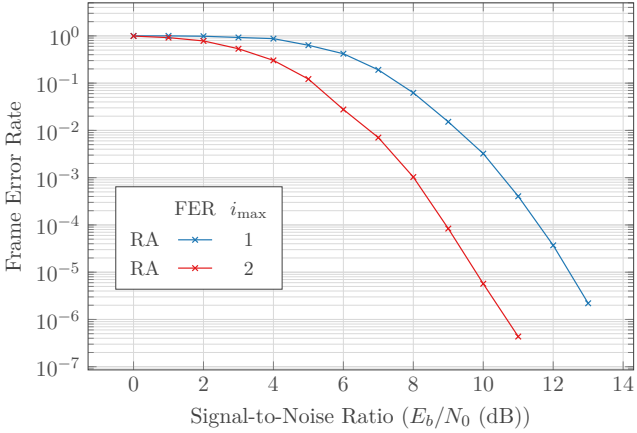
$K = 32$ RA code with random interleaver and 4 iterations.

Number of iterations depending on E_b/N_0



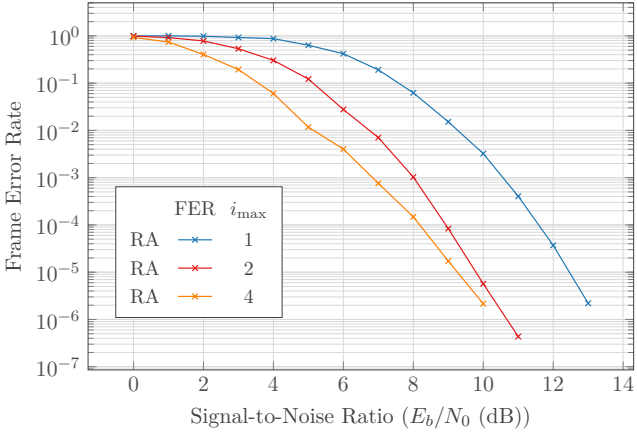
$K = 32, q = 4$ RA code with random interleaver.

Number of iterations depending on E_b/N_0



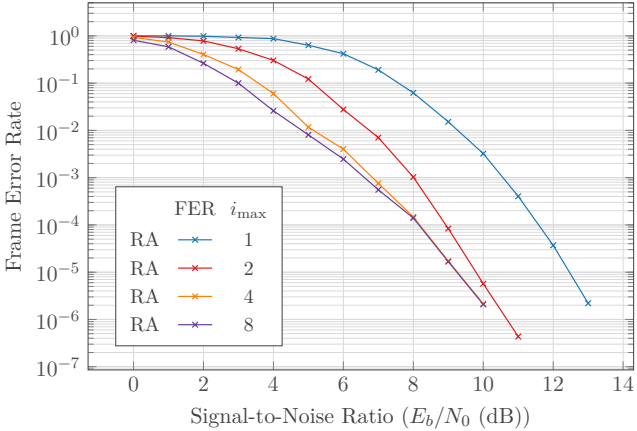
$K = 32, q = 4$ RA code with random interleaver.

Number of iterations depending on E_b/N_0



$K = 32, q = 4$ RA code with random interleaver.

Number of iterations depending on E_b/N_0



$K = 32, q = 4$ RA code with random interleaver.

Q&A

Thank you for listening!
Do you have any questions?

Bibliography

6 References

[1] Dariush Divsalar, Hui Jin, and Robert J. McEliece. "Coding Theorems for "Turbo-Like" Codes". In: *Allerton Conference on Communications, Control, and Computing*. 1998. URL: https://www.researchgate.net/profile/Dariush-Divsalar/publication/243768674_Coding_Theorems_for_Turbo-Like_Codes/links/53e3cc5c0cf2fb74870dbbc1/Coding-Theorems-for-Turbo-Like-Codes.pdf.

[2] C. Berrou, A. Glavieux, and P. Thitimajshima. "Near Shannon Limit Error-Correcting Coding and Decoding: Turbo-Codes". In: *International Conference on Communications (ICC)*. Vol. 2. IEEE, May 1993, pp. 1064–1070. DOI: 10.1109/ICC.1993.397441.

[3] R. Gallager. "Low-Density Parity-Check Codes". In: *IRE Transactions on Information Theory* 8.1 (Jan. 1962), pp. 21–28. DOI: 10.1109/TIT.1962.1057683.